Project - Low Level Design

on

Entertainment Planning Assistant Agent

Course Name: Agentic AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	AMAN SHARMA	EN22CS301109
2	ANANYA BHATIA	EN22CS301117
3	ANKIT NAGAR	EN22CS301134
4	ANUSHA SINGH PANWAR	EN22CS301179
5	ARPIT PATIDAR	EN22CS301199

Group Name: Group 02D2

Project Number: AAI-14

Industry Mentor Name:

University Mentor Name: Prof. Ajeet Singh Rajput

Academic Year: Jan-June 2026

Table of Contents

1.Introduction.
1.1 Scope of the Document

This document provides a detailed low-level design of the Agentic AI Entertainment Planning Assistant. It outlines the internal architecture, system components, data flow, and interaction between different modules of the system.

The scope includes:

- Design of the AI agent and its reasoning mechanism
- Description of tool integration and execution flow
- Data handling and memory usage
- API structure and communication between components

1.2 Intended Audience

This document is intended for:

- **Developers** – To understand system implementation and module-level design
- **System Architects** – To review design decisions and architecture
- **QA/Test Engineers** – To design test cases based on system flow
- **Project Evaluators/Faculty** – To assess the technical depth and design of the project

1.3 System Overview

The Agentic AI Entertainment Planning Assistant is an intelligent system designed to assist users in planning entertainment activities such as movie reviews, schedules, or content planning.

The system uses an **AI agent based on the ReAct (Reasoning + Acting) framework**, which allows it to:

- Understand user goals
- Break down tasks into smaller steps

- Dynamically call tools (mock APIs)
- Generate structured and actionable plans

The application provides a simple user interface where users input their goal, and the system processes it using an AI-powered backend to generate personalized entertainment plans.

Key features include:

- Goal-based planning using AI agents
- Tool-based execution (e.g., fetching movies, generating reviews)
- Step-by-step reasoning and decision-making
- Memory storage for context retention

The system is built using modern technologies such as **Streamlit (UI), Python (backend), LangChain (agent framework), and Gemini LLM**, making it scalable and extensible for future enhancements.

2. System Design

2.1 Application Design

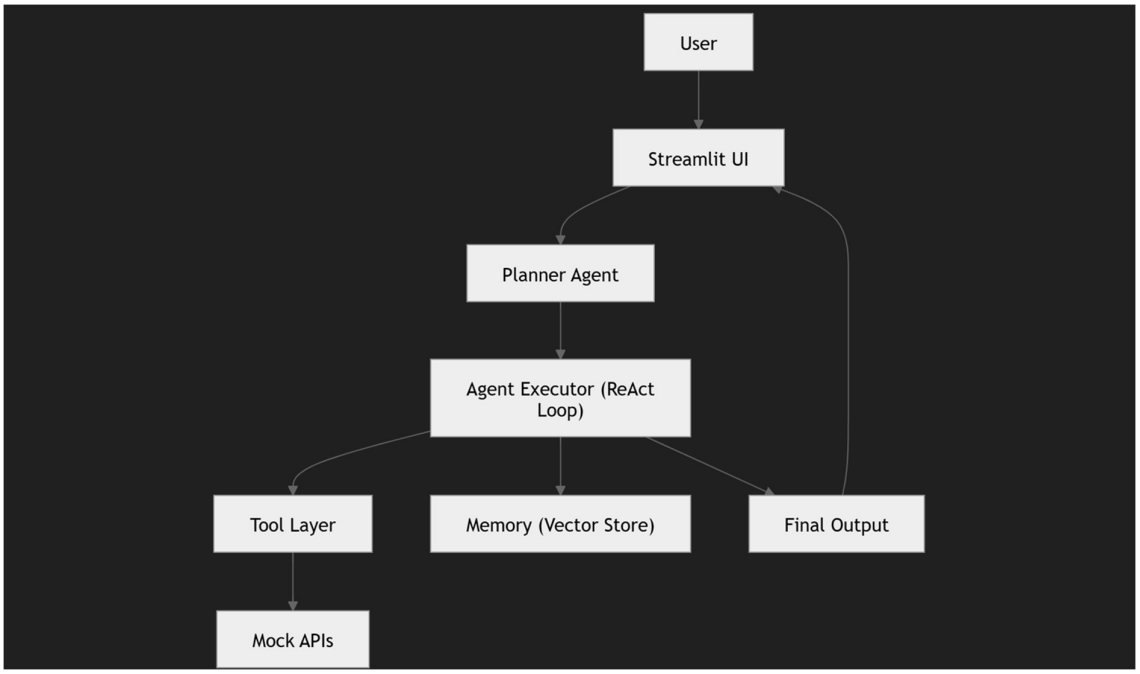
The *Agentic AI Entertainment Planning Assistant* follows a **layered, agent-based architecture** that integrates user interaction, intelligent reasoning, and tool execution.

Architecture Layers:

- **Presentation Layer (Streamlit UI)**
  - Accepts user input (goal)
  - Displays reasoning steps and final output
- **Agent Layer (Planner Agent)**
  - Implements **ReAct (Reason + Act)** paradigm
  - Uses LangChain's AgentExecutor
  - Controls decision-making and tool usage
- **Tool Layer (Mock APIs)**
  - Provides functional capabilities:

- Fetch trending movies
- Get movie metadata
- Generate reviews
- Create schedules

- **Memory Layer**
  - Stores past interactions
  - Uses vector embeddings for semantic retrieval



2.2 Process Flow

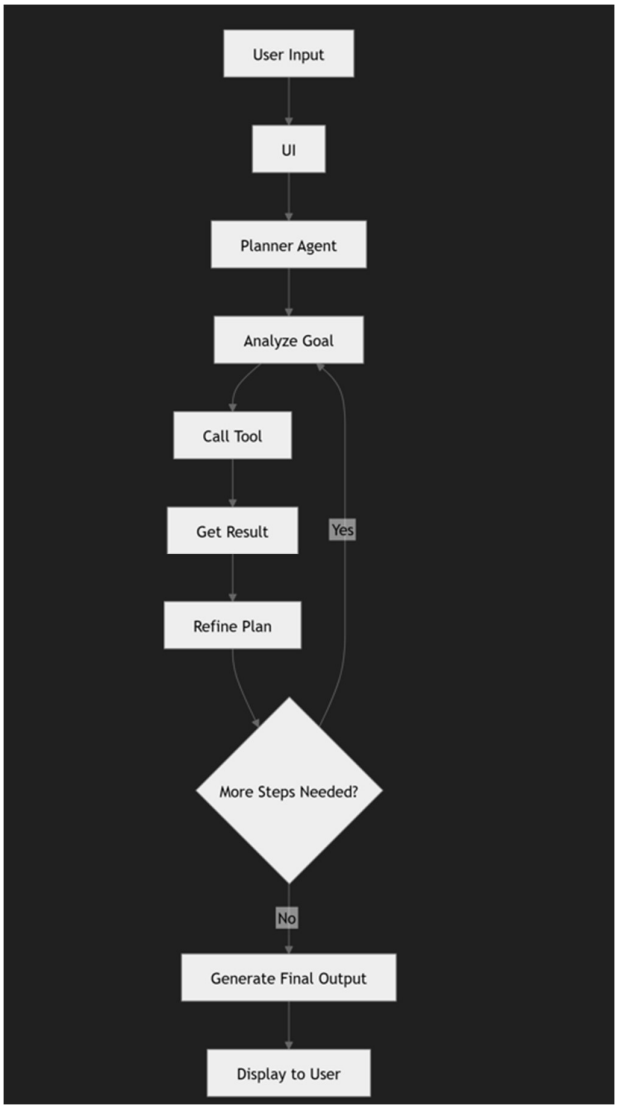
The system processes user requests through an intelligent agent loop.

Steps:

1. User enters a goal
2. UI sends input to backend
3. Planner Agent initializes
4. Agent enters ReAct loop:

- Think (analyze goal)
- Act (call tool)
- Observe (process result)

5. Tools return results
6. Agent refines plan
7. Final plan is generated
8. Output displayed to user
9. Data stored in memory



2.3 Information Flow

The system ensures structured movement of data between components.

Input:

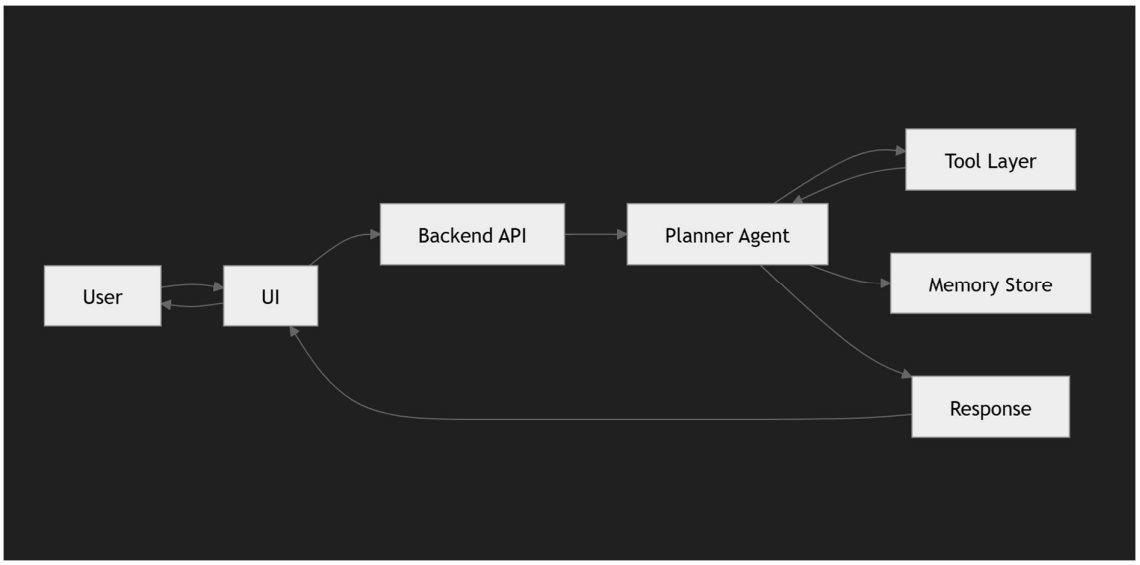
- User goal (text)

Processing:

- Natural language understanding
- Task decomposition
- Tool execution
- Iterative reasoning

Output:

- Structured entertainment plan



2.4 Components Design

The system is divided into modular components:

1. UI Module (Streamlit - app.py)

- Handles user interaction
- Displays logs and outputs

2. Planner Agent Module (planner_agent.py)

- Core intelligence
- Uses LangChain AgentExecutor
- Implements ReAct reasoning

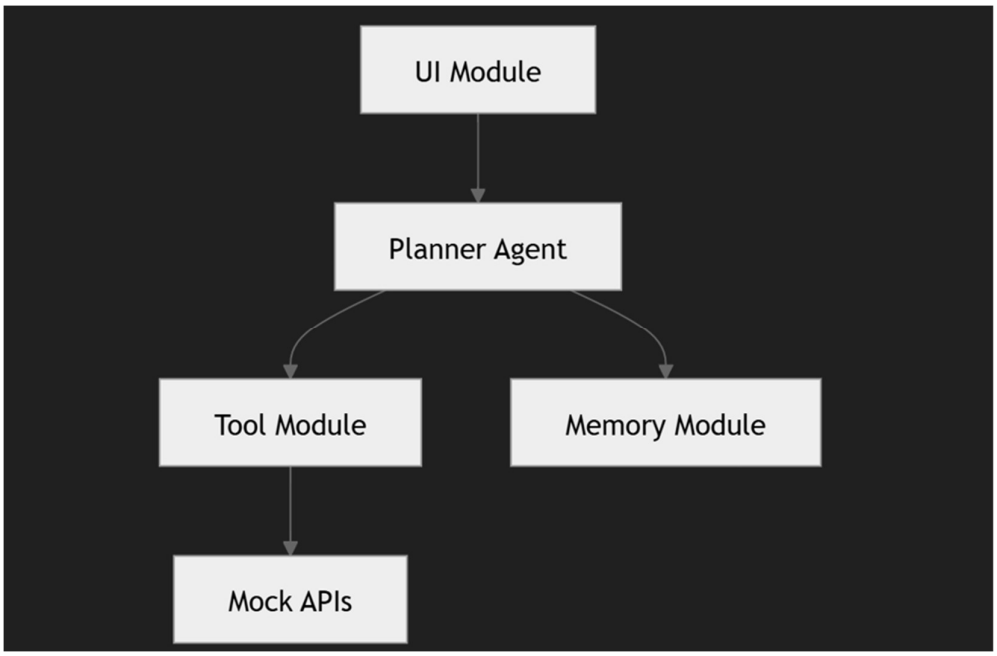
3. Tool Module (mock_tools.py)

Provides functionalities:

- fetch_trending_movies()
- get_movie_metadata()
- write_movie_review()
- generate_publishing_schedule()

4. Memory Module (vector_memory.py)

- Stores past interactions
- Uses embeddings for similarity search



2.5 Key Design Considerations

1. Agentic Reasoning

- Uses ReAct pattern for dynamic decision-making

2. Modularity

- Independent components (UI, agent, tools, memory)

3. Scalability

- Can integrate real APIs instead of mock tools

4. Performance

- Efficient reasoning using lightweight LLM (Gemini Flash)

5. Usability

- Simple and interactive Streamlit interface

6. Extensibility

- New tools and domains can be added easily

7. Maintainability

- Clean separation of logic and components

2.6 API Catalogue

Internal Tool APIs

API Name	Type	Description
fetch_trending_movies	Tool	Returns trending movies
get_movie_metadata	Tool	Fetches movie details
write_movie_review	Tool	Generates review content
generate_publishing_schedule	Tool	Creates final schedule

Agent API

API Name	Method Description
orchestrate_plan	Internal Executes full planning process

3. References

[1] LangChain, "LangChain Documentation," *LangChain*, 2026. [Online]. Available: https://docs.langchain.com/

[2] S. Yao *et al.*, "ReAct: Synergizing Reasoning and Acting in Language Models," *arXiv preprint arXiv:2210.03629*, 2022. [Online]. Available: https://arxiv.org/abs/2210.03629

[3] Google, "Gemini API Documentation," *Google AI*, 2026. [Online]. Available: https://ai.google.dev/

[4] Streamlit, "Streamlit Documentation," *Streamlit Inc.*, 2026. [Online]. Available: https://docs.streamlit.io/

[5] Python Software Foundation, "Python Documentation," 2026. [Online]. Available: https://docs.python.org/3/

[6] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," *Proceedings of EMNLP*, 2019. [Online]. Available: https://www.sbert.net/

[7] J. Johnson, M. Douze, and H. Jégou, "FAISS: A Library for Efficient Similarity Search," *Facebook AI Research*, 2017. [Online]. Available: https://faiss.ai/

[8] Lokesh Gupta, "REST API Design Best Practices," *RESTful API Tutorial*, 2026. [Online]. Available: https://restfulapi.net/